10-Day Django Bootcamp: Game Key Management & Publisher Webhooks

Course Description

Build a production-ready API for selling game keys. Publishers upload games, users buy keys, and **the system automatically notifies publishers via webhook when a game key expires**. Learn Django, DRF, async webhooks, and testing in 10 days (1.5h/day).

Prerequisites

- Basic Python, functions, classes, OOP
- HTTP basics, JSON

Tech Stack

Tool	Purpose
Python 3.12+	Runtime
`uv`	Package manager
Django 6.0	Web framework
DRF 3.17+	REST API
`python-decouple`	Config via `.env`
Celery + Redis	Async task queue
`pytest-django`	Testing
SQLite	Dev database

Deliverables

- GitHub repository with code, `.env` example, and README
- Working API: games, orders, key expiry detection
- Async webhook to publisher with retries and delivery logs
- pytest test suite (≥70% coverage on core)

Assessment

- Daily checkpoints: **20%**
- Final project: **80%**

🏫 Daily Syllabus (1.5 hours each day)

Day 1 – Environment & Project Setup

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure you have Python 3.12+ installed.
   - Open a clean terminal window in an empty project directory.
   - Have a web browser open and ready to navigate to `http://127.0.0.1:8000/`.
2. **Prerequisites Checklist**:
   - Students should have standard terminal environments working on their OS (Mac/Linux or Windows Git Bash/PowerShell).
3. **Required Material**:
   - Whiteboard or slide showing the end-to-end system architecture (User → Orders API → DB/ GameKey → Expiration Engine → Celery → Webhook Queue → Publisher).

🎯 Learning Objectives

By the end of this class, students will be able to:

- Install and configure modern Python package management using `uv`.
- Initialize and activate a virtual environment.
- Scaffold a new Django project and custom app.
- Securely configure environment variables using `python-decouple`.
- Initialize a Git repository and prevent secrets leaks using `.gitignore`.

- Run the Django development server and verify the default setup.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Interactive overview of the bootcamp outcome (game key expiry & publisher webhooks).
2. Key Concepts & Core Ideas	15 min	Introduction to MTV, API backends, package management (`uv`), and configuration security.
3. Live Coding Walkthrough	45 min	Live setup of python environment, Django, and decoupling configuration.
4. Check for Understanding	10 min	Q&A on virtual environments, secrets management, and DEBUG mode.
5. Class Wrap-up & Checkpoint	5 min	Verification of students' local environments.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Open a diagram showing the complete workflow. Explain the business problem: digital game key marketplaces need to deliver keys to users, track their expirations, and automatically notify publishers when a key expires so they can deactivate it on their servers.
- **Big Picture**: Show the finished flow: User buys key → key expires → Celery fires webhook → publisher receives it. Explain to students: *Make them care about the outcome before writing a single line.*

2. Key Concepts & Core Ideas (15 min)

Introduce the following definitions to the students:

- **What is Django?** MTV (Model-Template-View) framework. Mention that we'll use it purely as a REST API backend with no HTML templates.
- **What is DRF?** Django REST Framework. A toolkit built on top of Django that gives us serializers, viewsets, and routers for building RESTful APIs.
- **Virtual environments**: Why isolation matters (preventing dependency conflicts and ensuring reproducibility across developers' machines).
- **DEBUG=True vs False**: Django displays highly detailed error pages containing traceback information. Explain that this is crucial for development but a critical security leak in production.
- **Why python-decouple?** Reading from `os.environ.get()` is functional but `decouple` simplifies casting types (e.g. converting a string `"True"` to boolean `True`) and handling defaults and local `.env` files in a

clean API.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Tooling Installation & Scaffolding

- **Explain**: Contrast standard `pip` + `venv` with `uv`. Why are we using `uv`? Speed, lockfiles, and reproducibility. Run the installation and scaffolding commands while students follow along.
- **Code**:

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create and activate virtual environment
uv venv
source .venv/bin/activate

# Add dependencies
uv add django djangorestframework python-decouple celery redis pytest-django

# Scaffold project and app
django-admin startproject gamekey_platform .
python manage.py startapp games
```

- **Verify**: Confirm that every student sees `(.venv)` in their command line prompt. Pause here to ensure everyone is inside the active virtual environment before moving on.
- ⚠ **Common Pitfalls**:
  - **Forgetting to activate the venv**: If students skip `source .venv/bin/activate`, they will install dependencies globally or get "django not found". Teach them to run `which python` to verify the path points to `.venv`.
  - **Windows path differences**: Flag that on Windows, the activation command is `.venv\Scripts\activate` rather than `.venv/bin/activate`.

Step 3.2: Environment Variable Configuration

- **Explain**: Explain why secrets (such as the Django `SECRET_KEY`) should never go into version control (source code). Show what happens if you push a secret key to a public GitHub repository. Create the local environment file.
- **Code**: Create `.env` in the project root directory:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

- **Verify**: Check that the `.env` file is created and contains the correct key-value pairs.

Step 3.3: Django settings.py Decoupling

- **Explain**: Now we need to modify Django's settings to read our newly created `.env` file rather than having

hardcoded variables.

- **Code**: Update `gamekey_platform/settings.py` to import and use `python-decouple`:

```
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
```

- **Verify**: Run the development server to verify the configuration is loaded correctly:

```
# Verify setup
python manage.py runserver
```

Navigate to `http://localhost:8000/` and verify the Django welcome page loads.

- ⚠ **Common Pitfalls**:
  - ○ **`SECRET_KEY` still hardcoded**: Students might add `python-decouple` but forget to update the actual variables in `settings.py`. Test this by removing the key from `.env` and seeing if Django throws an error on startup.

Step 3.4: Git Initialization

- **Explain**: Set up version control. Introduce the `.gitignore` file and emphasize that the `.env` file must be ignored immediately to prevent leaking the secret keys.
- **Code**:

```
git init
```

Create `.gitignore` and include `.env` (along with standard Python/Django patterns: `__pycache__`, `.venv`, `*.pyc`, etc.):

```
.env
.venv/
*.pyc
__pycache__/
db.sqlite3
```

- **Verify**: Run `git status` and confirm that `.env` is NOT listed under untracked files.
- ⚠ **Common Pitfalls**:
  - ○ **Committing `.env` to Git**: Show the students what `git status` looks like *before* and *after* editing `.gitignore` to show how Git tracks files.

---

4. Check for Understanding (10 min)

> ***Question***: *"Why can't we just hard-code `SECRET_KEY = 'abc123'` in settings.py?"*
> ***Answer***: *Hard-coding `SECRET_KEY` in source code poses a severe security risk. If the repository is pushed to a public platform (like GitHub), anyone can see the key. Attackers can use it to forge signed cookies, session data, or password reset tokens. Storing it in an environment variable (`.env` file) keeps it private and allows different values for development and production environments.*

> ***Question***: *"What would happen if two projects on the same machine installed different versions of `django` without virtual environments?"*
> ***Answer***: *Installing different versions globally would cause a conflict because Python can only resolve one version of a package in its global site-packages directory. Installing a different version for the second project would overwrite the version needed by the first project, breaking it. Virtual environments isolate dependencies per project, allowing each to run its required version independently.*

> ***Question***: *"What does `DEBUG=True` change about how Django behaves?"*
> ***Answer***: *When `DEBUG=True`, Django displays detailed error traceback pages to the client (useful for debugging but a huge security leak in production because it exposes database queries, settings, and path variables). It also runs the built-in development server's auto-reloader and serves static/media files automatically. In production (`DEBUG=False`), it returns a generic 500 error page, requires `ALLOWED_HOSTS` to be set, and disables automated static file serving.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ Student shows the Django welcome rocket page running locally at `http://127.0.0.1:8000/`.
  - ○ The `.env` file exists and contains `SECRET_KEY` and `DEBUG`.
  - ○ Running `git status` shows `.env` is untracked (not staged) and excluded.
  - ○ Running `git log --oneline` shows at least one commit.

Day 2 – Core Models: Publisher, Game, GameKey

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is activated (`source .venv/bin/activate`).
   - Have the Django dev server stopped or running in the background.
   - Open `games/models.py` and `games/admin.py` in your IDE.

2. **Prerequisites Checklist**:
    - Students must have completed Day 1 (Django project scaffolded, custom `games` app created, and dev server working).

3. **Required Material**:
    - A diagram or whiteboard showing the database relationships: `User (Django built-in) <1-to-1> Publisher <1-to-many> Game <1-to-many> GameKey <many-to-1 (optional)> User (owner)`

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain Django's ORM mental model (classes/attributes mapped to tables/columns).
- Define relationships using `ForeignKey` and `OneToOneField`.
- Apply cascading rules on relationships using `on_delete=models.CASCADE`.
- Generate and run database migrations using `makemigrations` and `migrate`.
- Register models with Django's administrative interface and manage data through the admin UI.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	20 min	Discuss ORM mental models, design the entities, and draw relationship diagrams.
2. Key Concepts & Core Ideas	15 min	Define fields, relationship types, cascade deletion, and migrations.
3. Live Coding Walkthrough	40 min	Live-code the models, run database migrations, and configure administrative site.
4. Check for Understanding	10 min	Q&A on relationship choices, migrations, and model properties.
5. Class Wrap-up & Checkpoint	5 min	Verify models are editable in the admin UI.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (20 min)

- **Teacher Action**: Draw an entity-relationship diagram on the board. Explain that before writing code or routing APIs, we must build a strong foundation: the database schema.
- **ORM Mental Model**: Explain that Django models are Python classes, class attributes represent database columns, and instances of these classes represent database rows.
- **Relationships**: Explain how our entities connect:

- **Publisher** has a one-to-one link to Django's built-in **User** (for logging in).
- **Game** belongs to a single **Publisher** (foreign key).
- **GameKey** belongs to a single **Game** (foreign key), and optionally has a **User** owner (when purchased).

2. Key Concepts & Core Ideas (15 min)

Introduce these essential terms:

- **ORM (Object-Relational Mapper)**: Bridges the gap between OOP in Python and Relational SQL databases (SQLite for dev, PostgreSQL for prod).
- **`ForeignKey` VS `OneToOneField`**:
  - `ForeignKey` establishes a one-to-many relationship (one publisher can release many games).
  - `OneToOneField` enforces uniqueness (one user can represent only one publisher).
- **`on_delete=models.CASCADE`**: When a parent object is deleted, all child objects referencing it are deleted. Contrast this with `models.SET_NULL` or `models.PROTECT`. Ask students: *"What happens to keys if we delete a game? Cascade makes sense here."*
- **`null=True` VS `blank=True`**: `null` is database-level (allows database cells to be NULL), while `blank` is validation-level (allows empty inputs in forms/serializers).
- **Migrations**: Explain migrations as a version-control system for the database schema. Emphasize that they must be generated first and then applied.

---

3. Live Coding Walkthrough (40 min)

Step 3.1: Defining the Models

- **Explain**: Open `games/models.py`. We will define the three core models. Explain why `GameKey.owner` uses `null=True, blank=True` (keys exist before being sold and having an owner).
- **Code**: Modify `games/models.py`:

```
from django.db import models
from django.contrib.auth.models import User


class Publisher(models.Model):
    name = models.CharField(max_length=200)
    webhook_url = models.URLField()
    webhook_secret = models.CharField(max_length=64)
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    def __str__(self):
        return self.name


class Game(models.Model):
    title = models.CharField(max_length=200)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
```

```
    price = models.DecimalField(max_digits=6, decimal_places=2)

    def __str__(self):
        return self.title


class GameKey(models.Model):
    STATUS_CHOICES = [('active', 'Active'), ('expired', 'Expired')]

    key_string = models.CharField(max_length=50, unique=True)
    game = models.ForeignKey(Game, on_delete=models.CASCADE)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='active')
    expires_at = models.DateTimeField()
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True, blank=True)

    def __str__(self):
        return self.key_string
```

- **Verify**: Run `python manage.py check` to ensure there are no syntax or configuration errors in the models.
- ⚠ **Common Pitfalls**:
  - **Missing `__str__` method**: Show students how the admin UI looks if `__str__` is omitted—it defaults to unhelpful strings like "Publisher object (1)".
  - **Confusing `null` and `blank`**: Students often write `blank=True` but forget `null=True` on nullable database fields, which triggers database integrity errors when saving empty records.

Step 3.2: Database Migrations

- **Explain**: Scan models for modifications and create schema migration instructions, then execute them.
- **Code**:

```
python manage.py makemigrations
python manage.py migrate
```

- **Verify**: Open the generated migration file (e.g., `games/migrations/0001_initial.py`) and read its contents to the class so they understand how Django translates models into schemas.
- ⚠ **Common Pitfalls**:
  - **Forgetting `'games'` app in `INSTALLED_APPS`**: If the app is missing from `settings.py`, `makemigrations` will report "No changes detected".
  - **Running `migrate` before `makemigrations`**: Nothing will happen. Remind students of the two-step migration lifecycle.

Step 3.3: Registering Models in Django Admin

- **Explain**: To inspect and manage our models, we will register them with Django's built-in administration panel.
- **Code**: Modify `games/admin.py`:

```
from django.contrib import admin
from .models import Publisher, Game, GameKey

admin.site.register(Publisher)
admin.site.register(Game)
admin.site.register(GameKey)
```

- **Verify**: Create a superuser and log into the admin panel to test model registration:

```
# Create admin credentials
python manage.py createsuperuser
```

Run the server, navigate to `http://127.0.0.1:8000/admin`, log in, and ensure you can create/edit a `Publisher`, a `Game`, and a `GameKey` manually.

---

4. Check for Understanding (10 min)

> ***Question***: *"Why does `GameKey.owner` use `null=True, blank=True` but `GameKey.game` does not?"*
> ***Answer***: *`GameKey.owner` is optional because a game key can exist (e.g., pre-loaded by a publisher) before any user purchases it. Thus, the database field must allow `NULL` (`null=True`) and API/form validation must allow it to be empty (`blank=True`). Conversely, a `GameKey` must always belong to a specific game, so `GameKey.game` is mandatory and cannot be null.*

> ***Question***: *"What's the difference between `makemigrations` and `migrate`? What does each file represent?"*
> ***Answer***: *`makemigrations` scans model definitions for changes and creates new, numbered migration files (e.g., `0001_initial.py`), which are blueprints describing how to modify the database schema. `migrate` actually executes those blueprints against the database, applying the changes to create/modify tables. The migration files represent versioned history of the schema and must be committed to git.*

> ***Question***: *"If we delete a `Publisher`, what happens to their `Game` records? What about their `GameKey` records?"*
> ***Answer***: *Because `Game.publisher` uses `on_delete=models.CASCADE`, deleting a `Publisher` automatically cascades and deletes all related `Game` records. In turn, because `GameKey.game` also uses `on_delete=models.CASCADE`, deleting those `Game` records will automatically cascade and delete all associated `GameKey` records.*

> ***Question***: *"Why is `key_string` marked `unique=True`?"*

> ***Answer**: A game key represents a single, unique digital asset. If it were not marked `unique=True`, the database could accidentally store duplicate key strings, leading to the same key being issued to multiple users, violating licensing rules and causing validation collisions.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - `python manage.py migrate` completes with no errors.
  - Student is logged into the `/admin` interface.
  - Student can create a `Publisher` record, a `Game` record, and a `GameKey` record successfully using the admin UI.

Day 3 – DRF Basics: Game & Publisher APIs

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is active (`source .venv/bin/activate`).
   - Run the Django development server (`python manage.py runserver`).
   - Open a browser window ready to access the Django REST Framework browsable API at `http://127.0.0.1:8000/api/`.
2. **Prerequisites Checklist**:
   - Students must have completed Day 2 successfully (models created, migrated, and superuser configured).
3. **Required Material**:
   - Whiteboard or slide detailing HTTP Verbs (GET, POST, PATCH, PUT, DELETE) and how they correspond to CRUD operations.

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain REST architecture basics and HTTP methods.
- Define DRF Serializers to handle translation between JSON and Django database models.
- Implement ModelSerializers and control data visibility (e.g., hiding secrets).
- Create ModelViewSets to handle CRUD routes automatically.
- Register endpoints with DRF's `DefaultRouter` and interact with the browsable API.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Define RESTful services. Compare standard Django views with Django REST Framework (DRF).
2. Key Concepts & Core Ideas	15 min	Explain Serializers as translators, ViewSets vs APIViews, and routers.
3. Live Coding Walkthrough	45 min	Configure settings, write serializers and viewsets, wire URLs, and test the endpoints.
4. Check for Understanding	10 min	Q&A on serialization validation, viewset mappings, and security exclusions.
5. Class Wrap-up & Checkpoint	5 min	Verify GET and POST requests are functional in the browsable API UI.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Ask students: *"How does a mobile app or a separate frontend website talk to our Django database?"* Explain that they cannot access database models directly. We must expose standard HTTP endpoints (REST APIs) returning JSON data.
- **REST & DRF Overview**: Show how DRF streamlines this process:
  - **Serializers**: Handle validation and translation of Python models to and from JSON.
  - **ViewSets**: Standardize operations on resources (create, list, retrieve, update, delete).
  - **Routers**: Generate consistent, predictable URLs automatically.

2. Key Concepts & Core Ideas (15 min)

Introduce the core mechanics of DRF:

- **Serializer = Translator**: Converts complex Django model querysets/instances into Python datatypes that can be rendered to JSON (serialization) and parses incoming JSON payload data into validated Python dictionaries (deserialization).
- **`ModelSerializer`**: A shortcut class that auto-generates serializer fields matching the model fields.
- **ViewSet vs APIView**:
  - `APIView` requires manual handling of HTTP methods (e.g. implementing `get()` or `post()`).
  - `ViewSet` groups standard CRUD operations (`list`, `create`, `retrieve`, `update`, `destroy`) into a single class.
- **`DefaultRouter`**: Automatically generates clean URL patterns for all operations registered on a ViewSet.
- **Browsable API**: A built-in DRF feature that provides a web-based client interface for testing API endpoints during development.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Enable Rest Framework

- **Explain**: Register the DRF app and our custom `games` app in Django settings so Django loads them.
- **Code**: Add to `INSTALLED_APPS` in `gamekey_platform/settings.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'games',
]
```

Step 3.2: Writing Serializers

- **Explain**: Create `games/serializers.py` to translate our models. Emphasize why we exclude `webhook_secret` in `PublisherSerializer`—never return webhook secret keys in public API responses.
- **Code**: Create `games/serializers.py`:

```
from rest_framework import serializers
from .models import Game, Publisher


class GameSerializer(serializers.ModelSerializer):
    class Meta:
        model = Game
        fields = '__all__'


class PublisherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Publisher
        exclude = ('webhook_secret',)  # never expose the secret
```

- **Verify**: Run `python manage.py check` to make sure the serializers do not have syntax or naming mistakes.
- ⚠ **Common Pitfalls**:
  - **Exposing sensitive fields**: Remind students that using `fields = '__all__'` on models containing secret credentials leaks data. Always use `exclude` or explicitly list fields when dealing with secrets.

Step 3.3: Creating ViewSets

- **Explain**: Create `games/viewsets.py` to define the database queries and serializers that our endpoints will use.
- **Code**: Create `games/viewsets.py`:

```
from rest_framework import viewsets
from .models import Game, Publisher
from .serializers import GameSerializer, PublisherSerializer
```

```
class GameViewSet(viewsets.ModelViewSet):
    queryset = Game.objects.all()
    serializer_class = GameSerializer


class PublisherViewSet(viewsets.ModelViewSet):
    queryset = Publisher.objects.all()
    serializer_class = PublisherSerializer
```

- **Verify**: Ensure the queryset and serializer classes are imported correctly.
- ⚠ **Common Pitfalls**:
  - **Missing queryset or serializer**: If either is missing on `ModelViewSet`, Django will throw an `AssertionError` during startup.

Step 3.4: Wiring URLs via Router

- **Explain**: Initialize DRF's `DefaultRouter`, register the viewsets, and include the generated routes in the primary Django URL configurations.
- **Code**: Modify `gamekey_platform/urls.py`:

```
from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from games.viewsets import GameViewSet, PublisherViewSet

router = DefaultRouter()
router.register(r'games', GameViewSet)
router.register(r'publishers', PublisherViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
]
```

- **Verify**: Start the development server (`python manage.py runserver`), open `http://localhost:8000/api/` in your browser, and verify the DRF browsable API root page displays with links to `/api/games/` and `/api/publishers/`.
- ⚠ **Common Pitfalls**:
  - **`rest_framework` missing from INSTALLED_APPS**: The browsable API page will crash or render without CSS if DRF isn't registered in `settings.py`.
  - **Forgetting `include(router.urls)`**: If you do not include the router's URLs inside `urlpatterns`, the registered routes won't match any requests.

---

4. Check for Understanding (10 min)

> ***Question**: "What does a serializer do with incoming data before it reaches the database?"*
> ***Answer**: A serializer validates the structure and types of incoming data against defined field rules, checks for required fields, runs custom validation logic (e.g., checking if a value is valid), and parses the raw JSON input into a validated Python dictionary (`serializer.validated_data`).*

> ***Question**: "What's the difference between `GET /api/games/` and `GET /api/games/1/`? Which ViewSet methods handle each?"*
> ***Answer**: `GET /api/games/` retrieves a list of all games and is handled by the `list` method of `GameViewSet`. `GET /api/games/1/` retrieves the details of a single game with ID 1 and is handled by the `retrieve` method of `GameViewSet`.*

> ***Question**: "Why might you want `fields = ('id', 'title', 'price')` instead of `'__all__'`?"*
> ***Answer**: Explicitly defining fields improves security and API efficiency by ensuring that sensitive internal fields (like database flags, internal statuses, or relationship keys) are not leaked to the frontend, and reduces bandwidth by excluding unused fields.*

> ***Question**: "What would happen if we forgot to exclude `webhook_secret` from the publisher serializer?"*
> ***Answer**: The `webhook_secret` would be serialized and returned in public GET/POST responses. This would allow anyone viewing the API output to obtain the secret, enabling them to forge webhook requests and make unauthorized deliveries appear authentic to the publisher's system.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ Navigating to `GET /api/games/` returns `200 OK` with a JSON array.
  - ○ Sending a `POST` request to `/api/games/` with a valid payload (title, publisher, price) successfully creates a record and returns `201 Created`.
  - ○ Navigating to `/api/publishers/` returns the publisher list and does NOT contain `webhook_secret` in the JSON response payload.

Day 4 – Authentication & Permissions

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is active (`source .venv/bin/activate`).

- Run the Django dev server (`python manage.py runserver`).
- Have Postman, curl, or HTTPie open to make HTTP requests with custom headers.

2. **Prerequisites Checklist**:
   - Students must have completed Day 3 successfully (DRF viewsets and routers configured, browsable API accessible).

3. **Required Material**:
   - Whiteboard or slide mapping the difference between Authentication (who you are) and Authorization (what you can do).

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain different API authentication schemes (Session, Token, JWT) and why stateless Token Auth fits REST APIs.
- Configure DRF Token Authentication globally.
- Create custom object-level permission classes in DRF to protect resources from unauthorized modifications.
- Build a user registration endpoint that hashes passwords securely and returns authentication tokens.
- Make authenticated API requests using custom request headers.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Compare API authentication strategies. Explain why stateless token-based auth is best for REST backends.
2. Key Concepts & Core Ideas	15 min	Distinguish authentication vs authorization. Explain DRF request lifecycle and permission checks.
3. Live Coding Walkthrough	45 min	Configure Token Auth, create custom permissions, write the register view, and test the endpoints.
4. Check for Understanding	10 min	Q&A on object-level permissions, password hashing, and token header formats.
5. Class Wrap-up & Checkpoint	5 min	Verify registration generates a token and unauthenticated writes are blocked.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Ask: *"Right now, anyone in the world can run a POST request to add a game to our store. How do we secure our API so only authenticated publishers can edit games, and normal users can only view them?"*
- **Authentication Options**: Compare standard session authentication (cookies/browser-based) with token authentication (header-based) and JSON Web Tokens (JWT). Explain that for stateless web APIs, header-based Token Auth is simple, secure, and robust.

2. Key Concepts & Core Ideas (15 min)

Introduce these essential terms:

- **Authentication vs Authorization**:
  - *Authentication* answers: *"Who are you?"* (handled by TokenAuthentication).
  - *Authorization* answers: *"What are you allowed to do?"* (handled by PermissionClasses).
- **`IsAuthenticatedOrReadOnly`**: A global permission class that allows unauthenticated read operations (`GET`, `HEAD`, `OPTIONS`) but restricts write operations (`POST`, `PATCH`, `DELETE`) to authenticated users.
- **`BasePermission`**: The base class for custom permission logic. Override `has_permission` for request-level checks, `has_object_permission` for instance-level checks. Both must return `True` for access to proceed.
- **`SAFE_METHODS`**: Read-only HTTP methods: `GET`, `HEAD`, `OPTIONS`.
- **Password Hashing**: Emphasize that storing plaintext passwords is a major security vulnerability. Django hashes passwords using PBKDF2.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Enable Token Authentication

- **Explain**: Register Django REST Framework's authtoken module and run migrations to create the token tables.
- **Code**: Add `'rest_framework.authtoken'` to `INSTALLED_APPS` in `gamekey_platform/settings.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'rest_framework.authtoken',
    'games',
]
```

Run the database migrations:

```
python manage.py migrate
```

- **Verify**: Confirm that the table `authtoken_token` is successfully created in the database (you can check using the Django Admin dashboard or dbshell).

- ⚠ **Common Pitfalls**:
  - ○ **Missing migrations**: If you configure Token Auth but don't run `migrate`, Django will throw a database error (`no such table: authtoken_token`) on the first auth request.

Step 3.2: Configure Global Auth settings

- **Explain**: Update Django settings to enforce Token Authentication and `IsAuthenticatedOrReadOnly` permissions across all endpoints by default.
- **Code**: Add or update `REST_FRAMEWORK` settings in `gamekey_platform/settings.py`:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ],
}
```

- **Verify**: Try sending a `POST` request to `http://localhost:8000/api/games/` without an authentication header and verify that the API returns a `401 Unauthorized` status.

Step 3.3: Creating Custom Object-Level Permissions

- **Explain**: We want to make sure that a publisher can only update or delete *their own* games. We will create a custom permission class that checks the owner of the publisher profile.
- **Code**: Create `games/permissions.py`:

```
from rest_framework.permissions import BasePermission, SAFE_METHODS


class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.publisher.user == request.user
```

Now apply this custom permission to the `GameViewSet` by updating `games/viewsets.py`:

```
from rest_framework import viewsets
from .models import Game, Publisher
from .serializers import GameSerializer, PublisherSerializer
from .permissions import IsOwnerOrReadOnly


class GameViewSet(viewsets.ModelViewSet):
    queryset = Game.objects.all()
    serializer_class = GameSerializer
    permission_classes = [IsOwnerOrReadOnly]
```

```
class PublisherViewSet(viewsets.ModelViewSet):
    queryset = Publisher.objects.all()
    serializer_class = PublisherSerializer
```

- **Verify**: Ensure that the custom permission class imports and compiles without errors.
- ⚠ **Common Pitfalls**:
  - **`has_object_permission` not firing on lists**: Remind students that `has_object_permission` is only called for detail views (`retrieve`, `update`, `destroy`). It is not called for `list` or `create` requests.

Step 3.4: Building the User Registration Endpoint

- **Explain**: Create a view that allows new users to sign up, hashes their passwords, and immediately generates and returns an auth token. Since unregistered users don't have a token, we must explicitly bypass global permissions using `@permission_classes([AllowAny])`.
- **Code**: Add the registration function to `games/views.py`:

```
from django.contrib.auth.models import User
from rest_framework.authtoken.models import Token
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from rest_framework import status


@api_view(['POST'])
@permission_classes([AllowAny])
def register(request):
    username = request.data.get('username')
    password = request.data.get('password')
    if not username or not password:
        return Response({'error': 'Username and password required.'}, status=status.HTTP_400_B
    user = User.objects.create_user(username=username, password=password)
    token, _ = Token.objects.get_or_create(user=user)
    return Response({'token': token.key}, status=status.HTTP_201_CREATED)
```

Now wire the URL endpoint in `gamekey_platform/urls.py`:

```
from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from games.viewsets import GameViewSet, PublisherViewSet
from games.views import register

router = DefaultRouter()
router.register(r'games', GameViewSet)
router.register(r'publishers', PublisherViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
    path('api/register/', register),
```

```
]
```

- **Verify**: Send a `POST` request to `http://localhost:8000/api/register/` with a JSON payload containing `username` and `password`. Verify that it returns `201 Created` along with a hex token.
- ⚠ **Common Pitfalls**:
  - **Using `create()` instead of `create_user()`**: If you use `User.objects.create(...)`, Django stores the password in plaintext, meaning the user can never log in because Django's auth system expects a hashed password. Always use `create_user()`.
  - **Wrong Authorization Header Format**: DRF Token Auth expects the header format `Authorization: Token <token>` (with a space and the word `Token`). Confuses developers used to the `Bearer <token>` format.

4. Check for Understanding (10 min)

> ***Question**: "What's the difference between `has_permission` and `has_object_permission`? Give an example where you'd use each."*
> ***Answer**: `has_permission` checks if the user has access to the endpoint in general (checked at the start of the request, e.g., "Is the user logged in?"). `has_object_permission` is only called during detail views (retrieve/update/delete) to check access to a specific database object (e.g., "Is the logged-in user the owner of this game record?").*

> ***Question**: "Why do we use `create_user()` instead of `create()` when making User objects?"*
> ***Answer**: `create_user()` is a helper method on Django's User manager that handles password hashing using a secure algorithm (like PBKDF2). Using `create()` directly would write the password to the database as plaintext, exposing it to database administrators or security breaches, and preventing the user from logging in since Django's auth system expects a hashed password.*

> ***Question**: "If we didn't add `@permission_classes([AllowAny])` to the register view, what would happen when a new user tries to register?"*
> ***Answer**: The registration request would be blocked by the global permission default (`IsAuthenticatedOrReadOnly`), returning a `401 Unauthorized` or `403 Forbidden` response. Since a new user does not have a token yet, they would be unable to register.*

> ***Question**: "What does `IsAuthenticatedOrReadOnly` allow vs deny? Is it the right default for this API?"*
> ***Answer**: It allows anyone (authenticated or anonymous) to perform safe, read-only HTTP methods (`GET`, `HEAD`, `OPTIONS`), but restricts writing methods (`POST`, `PUT`, `PATCH`, `DELETE`) to authenticated users. It is an excellent default for this API because it keeps the game catalog publicly browsable while*

> *securing modifications and purchases.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - `POST /api/register/` returns a JSON response containing an authentication token.
  - `POST /api/games/` without an `Authorization` header returns a `401 Unauthorized` error.
  - `POST /api/games/` with the header `Authorization: Token <token>` successfully creates the game and returns `201 Created`.
  - `PATCH /api/games/1/` with a token belonging to a user *other* than the owner of the game's publisher returns `403 Forbidden`.

Day 5 – Orders API: Buying a Game Key

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Verify the virtual environment is active (`source .venv/bin/activate`).
   - Run the Django dev server (`python manage.py runserver`).
   - Open `games/models.py`, `games/views.py`, and `gamekey_platform/urls.py` in your IDE.
2. **Prerequisites Checklist**:
   - Students must have completed Day 4 successfully (Token Authentication is configured and the registration endpoint `/api/register/` is working).
3. **Required Material**:
   - Whiteboard or screen to draw the timeline of a race condition:
     - Request A starts → Queries active keys → Finds Key 1 (unowned)
     - Request B starts → Queries active keys → Finds Key 1 (unowned)
     - Request A updates Key 1 owner to User A and saves
     - Request B updates Key 1 owner to User B and saves (overwriting or double-allocating!)

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain what race conditions are and how they arise in concurrent applications.
- Use Django's `transaction.atomic()` context manager to bundle operations.
- Apply pessimistic locking using Django ORM's `select_for_update()`.
- Define models to represent orders and line items.

- Generate cryptographically random strings using Python's `uuid` library.
- Build a secure transactional order endpoint using Django view functions.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	20 min	Pose the double-allocation race condition problem. Draw the timeline on the board.
2. Key Concepts & Core Ideas	15 min	Explain transactions, pessimistic locking (`select_for_update`), and datetime arithmetic.
3. Live Coding Walkthrough	40 min	Write the order models, migrate, write the transactional API endpoint, wire URLs, and test.
4. Check for Understanding	10 min	Q&A on transactional rollbacks, locking scope, and UUID vs serial IDs.
5. Class Wrap-up & Checkpoint	5 min	Verify successful key purchase via Postman and database state changes.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (20 min)

- **Teacher Action**: Open the diagram board. Present this scenario: *"Two users hit the 'Buy' button for the last remaining key of a game at the exact same millisecond. If our server handles them concurrently, how do we prevent both users from being sold the same key?"*
- **Race Condition Motivation**: Draw the concurrent query flow. Explain that without locking, both requests read the database, find the key unowned, assign it to their respective user, and write back. User A gets a key, and User B gets the exact same key. This is a business catastrophe.

2. Key Concepts & Core Ideas (15 min)

Introduce these database principles:

- **`transaction.atomic()`**: Standardizes database transactions. Ensures that all SQL queries executed inside the context manager succeed together, or roll back completely if an error occurs.
- **Pessimistic Locking via `select_for_update()`**: Forces a row-level database lock on the matching records. Any other database query attempting to lock or edit those same rows must wait until our transaction completes.
- **`auto_now_add=True`**: Django field option that automatically sets the field value to the current datetime when the model is first created.
- **`uuid4()`**: Universally Unique Identifier. Explain that using sequential serial numbers (like `1`, `2`, `3`) for

digital license keys makes them trivial to guess and steal. We use random UUIDs instead.

- **Timezone-aware arithmetic**: Why we use `django.utils.timezone.now()` instead of Python's standard `datetime.now()` to prevent timezone conflicts.

3. Live Coding Walkthrough (40 min)

Step 3.1: Defining Order and OrderItem Models

- **Explain**: In order to track key purchases, we need an `Order` model representing the cart/invoice, and an `OrderItem` representing the specific keys purchased.
- **Code**: Add `Order` and `OrderItem` models to `games/models.py`:

```
class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    purchased_at = models.DateTimeField(auto_now_add=True)


class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    game_key = models.OneToOneField(GameKey, on_delete=models.CASCADE)
```

Run the migrations:

```
python manage.py makemigrations
python manage.py migrate
```

- **Verify**: Check that migrations apply cleanly. You can register these models in `games/admin.py` for visibility:

```
from .models import Order, OrderItem
admin.site.register(Order)
admin.site.register(OrderItem)
```

Step 3.2: Writing the Order Creation View

- **Explain**: Create the `create_order` API endpoint in `games/views.py`. This endpoint checks if a key is pre-loaded for the game. If it is, it locks the key using `select_for_update()`, updates its owner, and returns it. If no key is pre-loaded, it generates a fresh one on the fly. All of this runs inside an atomic transaction block.
- **Code**: Add the import statements and the `create_order` function to `games/views.py`:

```
import uuid
from datetime import timedelta
from django.utils import timezone
from django.db import transaction
from rest_framework.decorators import api_view, permission_classes
```

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from .models import Game, GameKey, Order, OrderItem


@api_view(['POST'])
@permission_classes([IsAuthenticated])
def create_order(request):
    game_id = request.data.get('game_id')
    if not game_id:
        return Response({'error': 'game_id is required.'}, status=status.HTTP_400_BAD_REQUEST)

    try:
        game = Game.objects.get(id=game_id)
    except Game.DoesNotExist:
        return Response({'error': 'Game not found.'}, status=status.HTTP_404_NOT_FOUND)

    with transaction.atomic():
        # Use select_for_update to prevent race conditions
        existing_key = (
            GameKey.objects.select_for_update()
            .filter(game=game, status='active', owner__isnull=True)
            .first()
        )

        if existing_key:
            # Assign existing unowned key
            key = existing_key
            key.owner = request.user
            key.save()
        else:
            # Generate a fresh key
            key = GameKey.objects.create(
                key_string=str(uuid.uuid4()).upper(),
                game=game,
                status='active',
                expires_at=timezone.now() + timedelta(days=30),
                owner=request.user,
            )

        order = Order.objects.create(user=request.user)
        OrderItem.objects.create(order=order, game_key=key)

    return Response({
        'order_id': order.id,
        'game': game.title,
        'key': key.key_string,
        'expires_at': key.expires_at,
    }, status=status.HTTP_201_CREATED)
```

- **Verify**: Check for syntax errors. Make sure that all models are imported correctly.
- ⚠ **Common Pitfalls**:

- ○ **Omitting** : If the database transaction is not atomic, the key could be locked but a crash later in the request (e.g. failing to create `OrderItem`) would leave the key assigned to the user without completing the order.
- ○ **Missing `owner__isnull=True` constraint**: If you forget to filter out keys that already have an owner, you will re-assign existing sold keys.

Step 3.3: Wiring the Endpoint URL

- **Explain**: Connect the order creation view to `/api/orders/` in Django routing.
- **Code**: Add `create_order` to `gamekey_platform/urls.py`:

```
from games.views import register, create_order

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
    path('api/register/', register),
    path('api/orders/', create_order),
]
```

- **Verify**: Send a POST request to `http://localhost:8000/api/orders/` with a valid `game_id` and the auth header `Authorization: Token <your_token>`. Verify it returns `201 Created` with the order information.
- ⚠ **Common Pitfalls**:
  - ○ **Missing authentication**: Forgetting to pass the `Authorization` header on the Postman/curl request will cause DRF to return a `401 Unauthorized` block.

---

4. Check for Understanding (10 min)

> ***Question**: "What would go wrong without `select_for_update()`?"*
> ***Answer**: Without `select_for_update()`, a race condition could occur if two concurrent requests query the database at the same time for an available key. Both would find the same unowned key, assign it to their respective user, and write it back. This results in the same game key being sold twice (double-allocation).*

> ***Question**: "When does `transaction.atomic()` roll back? What triggers a rollback?"*
> ***Answer**: It rolls back when an unhandled exception is raised within the context manager block. If any error (like a database constraint violation or a Python runtime exception) occurs before the block exits successfully, all changes made to the database within that block are discarded.*

> ***Question**: "Why do we filter with `owner__isnull=True`? What does `owner` represent for a key that no one has purchased yet?"*

> ***Answer***: *We filter for `owner__isnull=True` to retrieve a key that has not yet been bought by any user. For an unpurchased key, `owner` is `None` (represented as `NULL` in the database). Filtering this way prevents re-allocating an already purchased key.*

> ***Question***: *"Why use `uuid4()` to generate the key string instead of, say, `GameKey.objects.count() + 1`?"*
> ***Answer***: *`uuid4()` generates cryptographically random, unique identifiers that cannot be guessed by users, preventing attackers from predicting and stealing other game keys. Using `count() + 1` is sequential, exposing the total volume of keys and making them trivial to guess and exploit.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ Sending a POST request to `/api/orders/` with a valid `game_id` and authentication token returns `201 Created`.
  - ○ The JSON payload contains the generated key, `order_id`, and `expires_at` (set to 30 days in the future).
  - ○ Submitting a POST request without a token returns `401 Unauthorized`.
  - ○ Submitting a POST request with an invalid `game_id` returns `404 Not Found`.

Day 6 – Detecting Expired Keys (Management Command)

🏷 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is active (`source .venv/bin/activate`).
   - Run the Django dev server (`python manage.py runserver`).
   - Open your project directory in the IDE, ready to create new nested directories and python scripts.
2. **Prerequisites Checklist**:
   - Students must have completed Day 5 successfully (Order database tables migrated, and order creation view operational).
3. **Required Material**:
   - Whiteboard or slide showing standard command line task scheduling setups (such as cron tabs or background runners).

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain Django custom management commands and their use cases.
- Scaffold the specific package structure required for custom Django commands.
- Subclass `BaseCommand` to write command line utilities.
- Perform high-performance database batch updates using the ORM `.update()` method.
- Construct timezone-aware database queries using the `__lte` (less than or equal to) lookup.
- Configure cron jobs to run custom commands on a schedule.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Define background tasks. Explain when to use management commands vs views or tasks.
2. Key Concepts & Core Ideas	15 min	Explain `BaseCommand` methods, stdout redirection, batch ORM operations, and timezone filters.
3. Live Coding Walkthrough	40 min	Create directories and files, implement `check_expired_keys` logic, run manually, and discuss cron configurations.
4. Check for Understanding	10 min	Q&A on stdout vs print, `.update()` side effects, and crontab syntax.
5. Class Wrap-up & Checkpoint	5 min	Verify expired keys are successfully processed by running the command in the terminal.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Ask students: *"So far, we have built APIs that run when a user makes a request (like buying a key). But who updates a key when it expires? Does the user have to request an endpoint to expire it? No, the system must check in the background automatically."*
- **Management Commands**: Explain that Django commands are scripts that can be invoked via `python manage.py [command]`. They are perfect for background scripts triggered by the OS scheduler (like cron).

2. Key Concepts & Core Ideas (15 min)

Introduce these command line concepts:

- **`BaseCommand`**: The base class for all Django CLI scripts. Custom commands must subclass it and implement a `handle()` method.
- **Stdout and Stderr Redirection**: Why commands must print output using `self.stdout.write()` instead of standard Python `print()`. It allows output filtering and unit testing verification.

- **`self.style.SUCCESS()`**: Colorizes terminal logs (e.g. green for success, red for errors) for better visual feedback.
- **Bulk Database Updates**: Contrast looping over matching rows and calling `.save()` (creates `N` SQL queries) with `.update()` (creates `1` SQL query).
- **Timezone-aware datetime lookups**: Why we use `timezone.now()` is mandatory when querying timezone-aware fields, and how `expires_at__lte` translates to SQL `expires_at <= NOW()`.

---

3. Live Coding Walkthrough (40 min)

Step 3.1: Scaffolding the Directory Structure

- **Explain**: Django scans directories to discover custom commands. We must create a specific nested folder layout under our custom app. Emphasize that every directory must contain an empty `__init__.py` file so Python treats them as packages.
- **Code**: Create the folder tree in the terminal:

```
games/
  management/
    __init__.py
    commands/
      __init__.py
      check_expired_keys.py
```

- **Verify**: Confirm that both `__init__.py` files are created.
- ⚠ **Common Pitfalls**:
  - **Missing `__init__.py`**: If either package initialization script is missing, Django will fail to register the command, returning `Unknown command: 'check_expired_keys'`.

Step 3.2: Implementing the Command Logic

- **Explain**: Create `games/management/commands/check_expired_keys.py`. Write a command class that queries all active keys whose expiration datetime is in the past, changes their status to `expired`, and prints a count of affected keys.
- **Code**: Write the following content inside `games/management/commands/check_expired_keys.py`:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey


class Command(BaseCommand):
    help = 'Mark active keys whose expiry has passed as expired.'

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.filter(
            expires_at__lte=timezone.now(),
            status='active'
        )
```

```
            count = expired_keys.update(status='expired')
            self.stdout.write(self.style.SUCCESS(f'Expired {count} keys.'))
```

- **Verify**: Ensure the file compiles without syntax errors.

Step 3.3: Manual CLI Testing

- **Explain**: We will manually trigger a key expiration. Log into the Django Admin dashboard and create or modify a `GameKey` record so that its `expires_at` value is set to one hour in the past, and its status is set to `active`. Then, run the command.
- **Code**: Execute the management command:

```
python manage.py check_expired_keys
```

- **Verify**: Check the console output. It should output a success message (e.g. `Expired 1 keys.`). Reload the Admin page and verify the status of the expired key has changed to `Expired`.
- ⚠ **Common Pitfalls**:
  - **Using `datetime.now()` instead of `timezone.now()`**: If you use naive Python datetimes, Django will raise a `TypeError` due to comparing offset-naive and offset-aware datetimes.
  - **Queryset caching after updates**: Remind students that `.update()` modifies the rows in the database, but does not update active Python model objects in memory. If we reference the objects inside a cached queryset after calling `.update()`, they will still show the old status.

Step 3.4: Scheduling in Production via Cron

- **Explain**: To run this script every 15 minutes automatically on our production Linux server, we would add a line to our system crontab file pointing to the virtualenv python interpreter.
- **Code**: Open crontab config (via `crontab -e`) and add:

```
*/15 * * * * /path/to/.venv/bin/python /path/to/manage.py check_expired_keys
```

- **Verify**: Discuss the cron parameters: `*/15` means every 15 minutes, followed by hour, day of month, month, and day of week wildcards.

---

4. Check for Understanding (10 min)

> ***Question**: "Why do we use `self.stdout.write()` instead of `print()` inside a management command?"*
> ***Answer**: Using `self.stdout.write()` is best practice because it integrates with Django's internal logging systems and output redirection. In testing, it allows the output to be intercepted and asserted on (using `call_command`), whereas `print()` writes directly to the standard system stdout, making it harder to test or suppress.*

> ***Question***: *"What's the performance difference between `.update()` and looping + `.save()`? Are there situations where you'd prefer `.save()` anyway?"*
> ***Answer***: *`.update()` executes a single SQL `UPDATE` statement in the database, which is extremely fast and efficient for bulk operations. Looping and calling `.save()` executes a separate SQL query for each record (N queries), which is slow. However, you would prefer `.save()` if you need to trigger model `save()` overrides, pre/post-save signals, or validation, which `.update()` bypasses.*

> ***Question***: *"What does `expires_at__lte=timezone.now()` translate to in SQL?"*
> ***Answer***: *It translates to a `WHERE` clause: `WHERE expires_at <= 'CURRENT_TIMESTAMP_VALUE'`. This filters for records where the expiration datetime is less than or equal to the current time.*

> ***Question***: *"If we want this command to run every 15 minutes automatically, what are our options?"*
> ***Answer***: *1) Setup a system-level cron job that executes the command through the virtualenv python. 2) Use a Celery Beat task that runs the management command (or its logic) on a schedule. 3) Use cloud-specific schedulers (like Render Cron Jobs or AWS EventBridge) to trigger the command.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ The `check_expired_keys.py` script is placed under `games/management/commands/`.
  - ○ Creating a key with a past expiration date and running `python manage.py check_expired_keys` runs successfully.
  - ○ The script prints a green confirmation message to the terminal.
  - ○ The database records are updated from `active` to `expired` successfully.

Day 7 – Webhook Fundamentals (Synchronous)

🏷 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is active (`source .venv/bin/activate`).
   - Have the Django dev server stopped or running in the background.
   - Open a browser page to `https://webhook.site` to get a temporary test URL.
   - Open `games/webhooks.py` and `games/management/commands/check_expired_keys.py` in your IDE.
2. **Prerequisites Checklist**:
   - Students must have completed Day 6 successfully (Management command structured and working

locally to expire keys).

3. **Required Material**:
   - Whiteboard or slide detailing the HTTP polling vs. webhook callback model, and the HMAC signing algorithm: `HMAC(webhook_secret, JSON_body) = Signature`

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain the concept of Webhooks (HTTP callbacks) and contrast them with polling.
- Implement cryptographic payload signing using HMAC-SHA256.
- Enforce deterministic JSON serialization using `sort_keys=True` in `json.dumps()`.
- Use the `requests` library to make synchronous POST requests with custom timeouts and headers.
- Identify the core limitations of synchronous HTTP calls in management commands.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	20 min	Explain the webhooks model. Contrast polling vs. event-driven callbacks.
2. Key Concepts & Core Ideas	20 min	Explain payload verification, HMAC, constant-time comparison, and JSON key sorting.
3. Live Coding Walkthrough	30 min	Write the synchronous webhook helper, integrate it with the management command, and test.
4. Discussion & Limitations	10 min	Analyze the failures of synchronous webhooks (blocking, timeout delays, no retry engine).
5. Check for Understanding	10 min	Q&A on security signatures, response codes, and sync blockages.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (20 min)

- **Teacher Action**: Contrast Polling vs Webhooks using a real-world analogy: *"Imagine checking your physical mailbox every 5 minutes to see if a package arrived (polling) versus having a delivery person ring your doorbell when it arrives (webhook). Webhooks are push notifications for APIs."*
- **Security Risk**: Explain that since any client can send a POST request to the publisher's public webhook URL, the publisher needs a way to guarantee the request actually came from our marketplace. We solve this using cryptographic payload signing.

2. Key Concepts & Core Ideas (20 min)

Introduce these webhook security and network concepts:

- **HMAC-SHA256**: Hash-based Message Authentication Code. Both systems share a secret. We sign the payload bytes with this secret, producing a signature header. The publisher recalculates the signature using the same secret and matches them.
- **`sort_keys=True` in JSON**: JSON objects are unordered maps. If keys are serialized in different orders on the sender and receiver, the raw byte sequences will differ, causing the HMAC signatures to mismatch. Sorting keys makes serialization deterministic.
- **`hmac.compare_digest()`**: A utility that compares digests in constant time. This prevents timing attacks (where attackers deduce correct signature bytes by measuring tiny variations in execution time).
- **Outbound Timeouts**: Why we must always set a timeout (e.g. `timeout=5`) when calling external services. Without a timeout, a hanging remote server will cause our command line process to block indefinitely.
- **`raise_for_status()`**: A method that raises an HTTP error if the server returns a `4xx` or `5xx` error code.

---

3. Live Coding Walkthrough (30 min)

Step 3.1: Implementing the Webhook Sender

- **Explain**: Create `games/webhooks.py`. We will write a helper function that constructs the expiry event payload, signs it using HMAC-SHA256, sets the custom headers, and dispatches the synchronous POST request.
- **Code**: Create `games/webhooks.py`:

```
import hmac
import hashlib
import json
import requests


def send_expiry_webhook(publisher, game_key_str, game_title, expired_at):
    payload = {
        "event": "game_key.expired",
        "game_key": game_key_str,
        "game_title": game_title,
        "expired_at": expired_at.isoformat(),
    }

    secret = publisher.webhook_secret.encode()
    body = json.dumps(payload, sort_keys=True).encode()
    signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

    headers = {
        "Content-Type": "application/json",
        "X-Signature": f"sha256={signature}",
    }
```

```
    try:
        response = requests.post(publisher.webhook_url, json=payload, headers=headers, timeout
        response.raise_for_status()
    except Exception as e:
        print(f"[Webhook] Delivery failed for publisher {publisher.id}: {e}")
```

- **Verify**: Check that the module imports compile without syntax errors.
- ⚠ **Common Pitfalls**:
  - **Type errors with secrets**: Explain that `hmac.new()` requires bytes, not strings. Both the `secret` and the `body` must be `.encode()`ed to bytes before hashing.

Step 3.2: Updating the Management Command

- **Explain**: Modify `check_expired_keys.py` to trigger the synchronous webhook sender for every key we mark as expired.
- **Code**: Update `games/management/commands/check_expired_keys.py`:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.webhooks import send_expiry_webhook


class Command(BaseCommand):
    help = 'Mark expired keys and notify publishers synchronously.'

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publisher').filter(
            expires_at__lte=timezone.now(),
            status='active'
        )
        count = expired_keys.update(status='expired')

        for key in GameKey.objects.filter(status='expired').select_related('game__publisher'):
            send_expiry_webhook(key.game.publisher, key.key_string, key.game.title, key.expire

        self.stdout.write(f'Expired {count} keys (sync webhooks sent).')
```

- **Verify**:
  - Open `https://webhook.site` in a browser and copy the unique URL.
  - Create a publisher in your Django admin panel and paste this URL into their `webhook_url` field.
  - Expire a key in admin (by setting `expires_at` in the past and status to `active`).
  - Run `python manage.py check_expired_keys`.
  - Check the `webhook.site` logs to confirm that the HTTP POST request arrived with the correct JSON body and the `X-Signature` header.

---

4. Discussion & Limitations (10 min)

- **Teacher Action**: Open a discussion on the problems with this synchronous approach:
  1. **Thread Blocking**: The entire script blocks during the HTTP call.
  2. **Cascading Slowness**: If one publisher's server is slow, the script takes longer, delaying webhooks for all subsequent publishers.
  3. **No Resiliency (Retries)**: If a publisher's server is down (connection error), the webhook is lost forever.
  - *Preview: Explain that tomorrow we will solve these three issues using an asynchronous task queue (Celery + Redis).*

---

5. Check for Understanding (10 min)

> ***Question**: "A publisher claims they received a webhook from us but the payload was tampered with. How does HMAC protect against this?"*
> ***Answer**: HMAC uses a shared secret to generate a cryptographic signature of the request body. If any character in the payload is altered during transit, the receiver's computed signature will not match the `X-Signature` header, revealing that the payload was modified and should be rejected.*

> ***Question**: "Why do we pass `sort_keys=True` to `json.dumps()`? What could go wrong without it?"*
> ***Answer**: JSON keys are unordered. If we don't sort keys, the serialization format might change (e.g. dictionary keys serialized in a different order), resulting in a different byte sequence. This would produce a different HMAC signature, causing verification to fail even if the content remains identical.*

> ***Question**: "What happens to the management command if the publisher's server is down and we're making synchronous HTTP calls?"*
> ***Answer**: The management command thread will block, waiting for the HTTP request to timeout (e.g. 5 seconds per request). If many keys expired for that publisher, or multiple publishers are down, the script will take a very long time to complete and could cause other pending updates to stall.*

> ***Question**: "What's the difference between a 4xx and 5xx response from the publisher's webhook endpoint? Should we retry both?"*
> ***Answer**: A `4xx` response represents a client error (e.g., `400 Bad Request` or `404 Not Found`), suggesting our payload is invalid or the endpoint is misconfigured; retrying will likely fail again, so we shouldn't retry. A `5xx` response represents a server error (e.g., `500 Internal Server Error` or `503 Service Unavailable`), suggesting temporary publisher outage; we should retry these as the server might recover.*

6. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ The `send_expiry_webhook` function is implemented in `games/webhooks.py`.
  - ○ Running `python manage.py check_expired_keys` dispatches an HTTP POST request to the publisher's webhook URL.
  - ○ The HTTP request header contains a valid `X-Signature` starting with `sha256=`.
  - ○ The webhook payload arrives at `webhook.site` successfully.

Day 8 – Async Webhooks with Celery

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure you have Docker running on your system.
   - Run the Redis server in a container (`docker run -d -p 6379:6379 redis:7-alpine`).
   - Open a browser window ready to access the Django Admin panel.
   - Prepare two separate terminal tabs: one for running the management command, and one for running the Celery worker daemon.

2. **Prerequisites Checklist**:
   - Students must have completed Day 7 successfully (Synchronous webhook sending working with `requests.post`).

3. **Required Material**:
   - Whiteboard or slide showing the task queue broker architecture: `Producer (Management Command) → Message Broker (Redis Queue) → Worker (Celery Daemon) → database (Logs)`
   - Compare to a restaurant ticket system: the waiter (producer) drops tickets on the rail (broker), and the cook (worker) processes them asynchronously.

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain the role of a message broker (Redis) and a task runner (Celery) in async applications.
- Configure Celery inside a Django project environment.
- Create a `WebhookDeliveryLog` database audit trail.
- Build resilient asynchronous Celery tasks using automatic retries and exponential backoff calculations.
- Use `.delay()` to dispatch tasks asynchronously.
- Run and monitor Celery worker processes from the command line.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Define background worker models. Map the producer-broker-consumer relationship.
2. Key Concepts & Core Ideas	15 min	Explain shared tasks, bindings, exponential backoff, and late acknowledgments.
3. Live Coding Walkthrough	45 min	Start Redis, configure Celery settings, write the Audit Log model, implement the Celery task, update the CLI command, and run the worker.
4. Check for Understanding	10 min	Q&A on Redis message queue states, worker crashes, and task acknowledgment flags.
5. Class Wrap-up & Checkpoint	5 min	Verify async webhook tasks execute in the worker terminal and write logs to admin.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Open the queue diagram. Ask: *"What happens if a publisher's server goes down during our key check? In yesterday's code, the management command waited, blocked, and if the network request crashed, that webhook was lost forever. Today, we will throw the webhook job into a Redis buffer queue, let our management command finish instantly, and have a separate background process handle the HTTP delivery and automatic retries in the background."*
- **Decoupled Architecture**: Explain that by isolating the execution queue, if our web server crashes, tasks in Redis remain safe. If a publisher is offline, Celery will wait and try again later.

2. Key Concepts & Core Ideas (15 min)

Introduce these asynchronous concepts:

- **Task Queue** / **Broker**: An intermediary (Redis) that receives messages from the publisher and buffers them until a consumer (Celery worker) retrieves them.
- **`@shared_task`**: A decorator that defines a reusable Celery task without requiring explicit imports of the project-specific Celery application instance.
- **`bind=True`**: Binds the task function to the task instance, passing the task object itself as `self` (enabling us to invoke retries).
- **Exponential Backoff**: Instead of retrying immediately (when the remote server is likely still down), we double the wait time on each retry: `60 * (2 ** attempt)` (i.e. 60s, 120s, 240s).
- **Late Acknowledgment (`task_acks_late=True`)**: Enforces that the worker only tells Redis it completed the task *after* the function successfully executes. If the worker crashes mid-task, Redis re-queues the

task for another worker.

3. Live Coding Walkthrough (45 min)

Step 3.1: Starting the Redis Message Broker

- **Explain**: Celery needs a queue to store tasks. We will spin up a lightweight Redis instance in a background Docker container.
- **Code**:

```
docker run -d -p 6379:6379 redis:7-alpine
```

- **Verify**: Verify that Redis is listening by running `docker ps` or connecting via `redis-cli ping`.

Step 3.2: Configuring Celery inside Django

- **Explain**: Create the Celery configuration file in the settings directory, update package initialization files, and add broker URLs to Django settings.
- **Code**: Create `gamekey_platform/celery.py`:

```
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'gamekey_platform.settings')

app = Celery('gamekey_platform')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()
```

Update `gamekey_platform/__init__.py` to load Celery on startup:

```
from .celery import app as celery_app

__all__ = ('celery_app',)
```

Add these configurations to the bottom of `gamekey_platform/settings.py`:

```
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_TASK_ALWAYS_EAGER = False  # set True in tests to run tasks synchronously
```

- **Verify**: Run `celery -A gamekey_platform inspect ping` to ensure Celery can connect to Redis.

Step 3.3: Creating the Webhook Audit Log Model

- **Explain**: In production, we must keep a permanent log of all webhook notifications, retries, and delivery responses for auditing and debugging.

- **Code**: Add to :

```
class WebhookDeliveryLog(models.Model):
    game_key = models.CharField(max_length=50)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    payload = models.JSONField()
    response_status = models.IntegerField(null=True, blank=True)
    error_message = models.TextField(blank=True)
    attempt = models.IntegerField(default=0)
    success = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        status = 'OK' if self.success else 'FAIL'
        return f"[{status}] {self.game_key} attempt #{self.attempt}"
```

Run the database migrations and register the model:

```
python manage.py makemigrations
python manage.py migrate
```

Register the model in `games/admin.py`:

```
from .models import WebhookDeliveryLog
admin.site.register(WebhookDeliveryLog)
```

- **Verify**: Log into Django Admin and verify that the `Webhook Delivery Logs` section is visible.

Step 3.4: Writing the Celery Task with Retries

- **Explain**: Create `games/tasks.py`. This task retrieves the publisher settings, builds the payload containing the current attempt count, signs it, and dispatches the POST request. If the HTTP call fails or raises an error, the task registers a failure log and schedules a retry with exponential backoff.
- **Code**: Create `games/tasks.py`:

```
import hmac
import hashlib
import json
import requests
from celery import shared_task
from .models import Publisher, WebhookDeliveryLog


@shared_task(bind=True, max_retries=3, default_retry_delay=60)
def send_expiry_webhook_async(self, publisher_id, game_key_str, game_title, expired_at_iso,
    publisher = None
    payload = {}
```

```
    try:
        publisher = Publisher.objects.get(id=publisher_id)

        payload = {
            "event": "game_key.expired",
            "game_key": game_key_str,
            "game_title": game_title,
            "expired_at": expired_at_iso,
            "attempt": attempt,
        }

        secret = publisher.webhook_secret.encode()
        body = json.dumps(payload, sort_keys=True).encode()
        signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

        headers = {
            "Content-Type": "application/json",
            "X-Signature": f"sha256={signature}",
        }

        response = requests.post(
            publisher.webhook_url,
            json=payload,
            headers=headers,
            timeout=5,
        )

        WebhookDeliveryLog.objects.create(
            game_key=game_key_str,
            publisher=publisher,
            payload=payload,
            response_status=response.status_code,
            attempt=attempt,
            success=response.status_code < 400,
        )

        if response.status_code >= 400:
            raise Exception(f"HTTP {response.status_code} from publisher endpoint")

    except Exception as exc:
        if publisher:
            WebhookDeliveryLog.objects.create(
                game_key=game_key_str,
                publisher=publisher,
                payload=payload,
                error_message=str(exc),
                attempt=attempt,
                success=False,
            )

        # Exponential backoff: 60s, 120s, 240s
        raise self.retry(exc=exc, countdown=60 * (2 ** attempt))
```

- ⚠ **Common Pitfalls**:

ⓘ **Omitting `on retry`**: If you call `` without raising it, the function execution doesn't stop, which can result in double logs. Always write `raise self.retry()`.

Step 3.5: Updating the Management Command

- **Explain**: Update `check_expired_keys.py` to dispatch tasks to the queue using `.delay()` instead of calling the function synchronously. Note that because we update the status field in bulk via `.update()`, we must materialize the queryset to a list *before* running the update.
- **Code**: Update `games/management/commands/check_expired_keys.py`:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.tasks import send_expiry_webhook_async


class Command(BaseCommand):
    help = 'Mark expired keys and dispatch async webhook notifications.'

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publisher').filter(
            expires_at__lte=timezone.now(),
            status='active'
        )

        # Capture before update() clears the queryset
        keys_to_notify = list(expired_keys)
        count = expired_keys.update(status='expired')

        for key in keys_to_notify:
            send_expiry_webhook_async.delay(
                publisher_id=key.game.publisher.id,
                game_key_str=key.key_string,
                game_title=key.game.title,
                expired_at_iso=key.expires_at.isoformat(),
                attempt=0,
            )

        self.stdout.write(self.style.SUCCESS(
            f'Expired {count} keys. Webhook tasks dispatched to Celery.'
        ))
```

Step 3.6: Running the Asynchronous Stack

- **Explain**: Run the Celery worker process and trigger the expiration flow to watch the system in action.
- **Code**: In terminal tab 1, start the background worker:

```
celery -A gamekey_platform worker --loglevel=info
```

In terminal tab 2, trigger the expiration check command:

```
python manage.py check_expired_keys
```

- **Verification Criteria**:
  - ○ The Celery background worker starts cleanly.
  - ○ Running `check_expired_keys` runs instantly without blocking for HTTP calls.
  - ○ The Celery worker console outputs task receipt and execution details.
  - ○ The `WebhookDeliveryLog` entries populate in the admin panel showing accurate response statuses and attempt numbers.

Day 9 – Testing with pytest-django

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is active (`source .venv/bin/activate`).
   - Clean up any previous test runs.
   - Open `pytest.ini`, `games/tests/test_webhook.py`, and `games/tasks.py` in your IDE.
2. **Prerequisites Checklist**:
   - Students must have completed Day 8 successfully (Celery worker, Redis queue, and database logging are configured and working).
3. **Required Material**:
   - Whiteboard or slide detailing the Testing Pyramid (Unit tests → Integration tests → End-to-End tests) and explaining what a Mock double is.

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain unit testing philosophies and what to mock vs. what to test.
- Install and configure `pytest`, `pytest-django`, and `pytest-mock` packages.
- Define reusable database fixtures with pytest.
- Inject database permissions in tests using `@pytest.mark.django_db`.
- Mock external network requests and Celery delay queues using `@patch`.
- Run a test coverage report and read its results to find untested paths.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Define the value of automated testing. Discuss why we mock network and clock dependencies.

2. Key Concepts & Core Ideas	15 min	Explain pytest fixtures, transactional db isolation, mocks, patches, and code coverage.
3. Live Coding Walkthrough	45 min	Install testing plugins, write configuration files, write fixtures, build three custom tests, run tests, and inspect coverage.
4. Check for Understanding	10 min	Q&A on mocking boundaries, `side_effect` vs `return_value`, and coverage reports.
5. Class Wrap-up & Checkpoint	5 min	Run test command and verify all tests pass with ≥70% coverage.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Ask the students: *"If we make a change to our order code on Day 12, how do we know we didn't break our Day 8 webhook logic? Do we start our web server, run Redis, spin up a Celery worker, and manually create an order every time? No, we write automated scripts that test all pathways in under 2 seconds."*
- **Mocking External I/O**: Explain that real tests shouldn't hit real external publisher servers, because if a publisher is down, our internal tests will fail. We use **mocking** to simulate the network response.

2. Key Concepts & Core Ideas (15 min)

Introduce these testing concepts:

- **Unit vs Integration Testing**:
  - *Unit test* verifies a single function in isolation (mocking all external dependencies).
  - *Integration test* checks if multiple modules work together (like Django views talking to the database).
- **`@pytest.fixture`**: Reusable functions that setup state before a test executes and tear down state afterward.
- **`@pytest.mark.django_db`**: By default, pytest blocks database calls to keep tests fast and isolated. This decorator opens up a temporary database transaction which rolls back at the end of the test.
- **`@patch('module.name')`**: Replaces an import reference with a dummy `MagicMock` object for the duration of the test. Emphasize: *Always patch where the module is imported and used, not where it is defined.*
- **Code Coverage**: The percentage of code lines executed during testing. Explain that high coverage indicates that lines were executed, but does not guarantee bug-free logic.

3. Live Coding Walkthrough (45 min)

Step 3.1: Package Installation & Configuration

- **Explain**: Install our testing libraries. We must tell `pytest` where our Django settings file is located using a local configuration file.
- **Code**: Install plugins:

```
uv add pytest-django pytest-mock
```

Create `pytest.ini` in the project root directory:

```
[pytest]
DJANGO_SETTINGS_MODULE = gamekey_platform.settings
python_files = test_*.py
```

Create an empty init file under `games/tests/` directory to mark it as a test suite:

```
games/tests/__init__.py
```

- **Verify**: Verify that running `pytest` from the terminal scans for tests, even if it finds zero.

Step 3.2: Writing the Fixtures

- **Explain**: We will create reusable data fixtures in `games/tests/test_webhook.py` that set up a publisher profile and an expired game key inside the temporary test database.
- **Code**: Create `games/tests/test_webhook.py`:

```
import pytest
from django.contrib.auth.models import User
from django.utils import timezone
from datetime import timedelta
from unittest.mock import patch, MagicMock
from django.core.management import call_command
from games.models import Publisher, Game, GameKey, WebhookDeliveryLog


@pytest.fixture
def publisher_user(db):
    user = User.objects.create_user(username='pub_user', password='pass')
    publisher = Publisher.objects.create(
        name='Test Publisher',
        webhook_url='https://example.com/webhook',
        webhook_secret='supersecret',
        user=user,
    )
    return publisher


@pytest.fixture
def expired_game_key(db, publisher_user):
    game = Game.objects.create(
        title='Epic Game',
        publisher=publisher_user,
```

```
        price='29.99',
    )
    return GameKey.objects.create(
        key_string='TEST-KEY-0001',
        game=game,
        status='active',
        expires_at=timezone.now() - timedelta(hours=1),
        owner=publisher_user.user,
    )
```

- **Verify**: Ensure that these fixtures load without syntax errors.

Step 3.3: Writing the Test Logic

- **Explain**: Add three unit tests to verify:
  1. The expiration management command successfully detects expired keys and dispatches tasks to the Celery queue.
  2. The Celery task logs a successful result inside the audit table when the publisher's HTTP response is `200`.
  3. The Celery task logs a failure entry inside the audit table when requests raise connection errors.
- **Code**: Append the tests to `games/tests/test_webhook.py`:

```
@pytest.mark.django_db
@patch('games.tasks.send_expiry_webhook_async.delay')
def test_management_command_dispatches_tasks(mock_delay, expired_game_key):
    call_command('check_expired_keys')
    assert mock_delay.called
    call_args = mock_delay.call_args
    assert call_args.kwargs['game_key_str'] == 'TEST-KEY-0001'


@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_success(mock_post, publisher_user, expired_game_key):
    from games.tasks import send_expiry_webhook_async
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_post.return_value = mock_response

    send_expiry_webhook_async(
        publisher_id=publisher_user.id,
        game_key_str='TEST-KEY-0001',
        game_title='Epic Game',
        expired_at_iso=expired_game_key.expires_at.isoformat(),
        attempt=0,
    )

    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is True
    assert log.response_status == 200
```

```
@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_failure(mock_post, publisher_user, expired_game_key):
    from games.tasks import send_expiry_webhook_async
    mock_post.side_effect = Exception('Connection refused')

    with pytest.raises(Exception):
        send_expiry_webhook_async.apply(kwargs=dict(
            publisher_id=publisher_user.id,
            game_key_str='TEST-KEY-0001',
            game_title='Epic Game',
            expired_at_iso=expired_game_key.expires_at.isoformat(),
            attempt=0,
        ))

    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is False
    assert 'Connection refused' in log.error_message
```

- **Verify**: Run `pytest` inside the terminal and ensure all tests run and pass.
- ⚠ **Common Pitfalls**:
  - **Wrong `@patch` target**: Remind students that patching `requests.post` globally does not work. We must patch `games.tasks.requests.post` because that's where the request is executed.
  - **Calling `.delay()` inside tests**: Calling `.delay()` would push the task onto a real Redis broker. For testing, we mock `.delay` (Test 1) or run the function synchronously using `task.apply(...)` (Test 3).

Step 3.4: Generating Coverage Reports

- **Explain**: Measure our test coverage to see what percentage of code lines were touched during execution.
- **Code**: Run coverage:

```
pytest --cov=games --cov-report=term-missing
```

- **Verify**: Review the command output. Highlight any lines marked missing and explain how to write tests targeting them.

---

4. Check for Understanding (10 min)

> ***Question**: "Why do we `@patch('games.tasks.requests.post')` rather than `@patch('requests.post')`?"*
> ***Answer**: We patch where the module is imported and used, not where it is defined. In `games/tasks.py`, the code uses `requests.post`. If we patched `requests.post` globally, `games/tasks.py` would still use its local, unpatched reference to the real `requests.post` function.*

> ***Question***: *"What does `mock_post.side_effect = Exception('Connection refused')` do differently from `mock_post.return_value = ...`?"*
> ***Answer***: *`.return_value` makes the mock function return a specific value (like a mock response object) when called. `.side_effect` raises the specified exception when the mock function is called, allowing us to test how the code handles connection failures or timeouts.*

> ***Question***: *"If a test has 100% line coverage, does that mean it's bug-free? Why or why not?"*
> ***Answer***: *No, 100% line coverage only means every line of code was executed at least once during the tests. It does not verify different combinations of inputs, edge cases, race conditions, logical errors, or unexpected database states.*

> ***Question***: *"Why do we use `task.apply(kwargs=...)` instead of `task.delay(...)` in the failure test?"*
> ***Answer***: *`task.apply()` runs the task synchronously in the current process, making it easy to catch exceptions and test the task logic step-by-step. `task.delay()` sends the task to Redis asynchronously, making it run in a separate Celery worker process where exceptions are caught by Celery and not raised in the test execution context.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - Running `pytest` runs and passes all 3 tests.
  - Running `pytest --cov=games --cov-report=term-missing` outputs a report showing test coverage of at least 70% for the custom app logic.

Day 10 – Final Integration & Deployment

📋 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure Docker and Docker Compose are installed and running locally.
   - Set up an active Git repository pushed to a private or public GitHub account.
   - Open a browser page to `https://render.com` (or your preferred cloud provider).
   - Open `Dockerfile`, `docker-compose.yml`, and `gamekey_platform/settings.py` in your IDE.
2. **Prerequisites Checklist**:
   - Students must have completed Day 9 successfully (All tests passing locally, coverage ≥ 70%).
3. **Required Material**:

- Diagram comparing Virtual Machines vs. Containers (highlighting host OS sharing).
- A diagram of the three containerized services and how they bind and network together: `Host Machine (Port 8000) → Web Container (Port 8000) ↔ Redis Container (Port 6379) ↔ Worker Container`

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain containerization principles and how Docker solves environment dependency issues.
- Build a Python base image using `Dockerfile` instructions and the `uv` tool.
- Assemble multi-container architectures using `docker-compose.yml`.
- Configure binding and ports to expose applications to local hosts.
- Configure and deploy applications to Render (Web Services, Workers, and Redis).
- Complete a final system integration test checking endpoints, auth, tasks, audit logs, and tests.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	20 min	Introduce containerization. Discuss local vs. production differences.
2. Key Concepts & Core Ideas	15 min	Explain Docker images vs. containers, port bindings, compose networks, and production web servers.
3. Live Coding Walkthrough	45 min	Write the Dockerfile, write docker-compose.yml, test multi-service stack locally, and deploy services to Render.
4. Check for Understanding	10 min	Q&A on build vs. runtime commands, local loops in containers, and database ephemeral disks.
5. Class Wrap-up & Wrap-up Checklist	10 min	Complete final system integration checklist. Celebrate bootcamp completion!

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (20 min)

- **Teacher Action**: Ask students: *"How many times have you heard 'but it works on my machine!' when deploying code? Today, we will package our application code, virtual environment, and system packages into an immutable container image. This image will run identically on your laptop, a colleague's machine, or a cloud server in Oregon."*
- **Multi-Service Architecture**: Review the final architecture. To run this app, we need three pieces: a web API server, a Redis queue, and a Celery worker. Running these individually on a server is complex;

Docker Compose simplifies this by orchestrating them in a single command.

2. Key Concepts & Core Ideas (15 min)

Introduce these container and deployment rules:

- **Docker Image vs. Container**: An *image* is the read-only blueprint (like a class definition). A *container* is a running, writable instance of that image (like an instantiated object).
- **`RUN` vs. `CMD`**:
  - `RUN` executes during the image build phase (installing tools, running compilations).
  - `CMD` defines the default shell script execution command when the container starts up.
- **`0.0.0.0` Binding**: Django's dev server binds to `127.0.0.1` by default. Inside a container, `127.0.0.1` is completely isolated. Binding to `0.0.0.0` tells the container to listen for network traffic coming from outside its boundaries.
- **Service Networking**: In Docker Compose, containers communicate using service names (e.g. `redis://redis:6379/0`) rather than `localhost`.
- **Gunicorn**: Explain that Django's `runserver` is single-threaded and unsafe for production. In production, we swap `runserver` for a WSGI HTTP server like `Gunicorn`.
- **Ephemeral Filesystem**: Explain that cloud containers are stateless; any file written to local disk (like `db.sqlite3`) is destroyed on redeploy. Thus, real databases (PostgreSQL) must run outside the web container.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Writing the Dockerfile

- **Explain**: Create the image blueprint. We start from a slim Python 3.12 image, copy `uv` binary to install dependencies rapidly, copy our code, sync libraries, and set the default launch command.
- **Code**: Create `Dockerfile` in the project root:

```
FROM python:3.12-slim

# Install uv
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv

WORKDIR /app
COPY . .

RUN uv sync --no-dev

CMD ["uv", "run", "python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Step 3.2: Assembling the Multi-Container Compose Stack

- **Explain**: Create the compose file to wire up our web, worker, and Redis services. Notice that both `web` and `worker` build from the same Dockerfile, but execute different commands on startup.

- **Code**: Create in the project root:

```
version: '3.9'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - redis
    command: uv run python manage.py runserver 0.0.0.0:8000

  worker:
    build: .
    env_file:
      - .env
    depends_on:
      - redis
    command: uv run celery -A gamekey_platform worker --loglevel=info

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

- **Verify**: Ensure YAML indentation is correct.

Step 3.3: Running the Stack Locally

- **Explain**: Compile the images and bring the services up.
- **Code**:

```
docker compose up --build
```

- **Verify**: Watch the terminal output. Confirm that `web`, `worker`, and `redis` boot up without errors. Send a POST request to `/api/orders/` and verify that tasks process in the containerized worker terminal.
- ⚠ **Common Pitfalls**:
  - **`CELERY_BROKER_URL` misconfiguration**: Ensure that inside Docker Compose, `CELERY_BROKER_URL` points to `redis://redis:6379/0` (the name of the container service), not `localhost`.

Step 3.4: Deploying to Render

- **Explain**: Walk students through pushing the repository to GitHub and deploying to Render.
- **Steps**:
  1. Push code to a GitHub repo.
  2. Create a new **Web Service** on Render, pointing to your repo.
     - Build Command: `uv sync --no-dev && python manage.py migrate`

- Start Command:

3. Create a **Redis** instance on Render and copy its connection URL.
4. Inject Environment Variables into the Web Service:
   - `CELERY_BROKER_URL` (pasted Redis URL)
   - `SECRET_KEY` (secret token)
   - `DEBUG` (`False`)
   - `ALLOWED_HOSTS` (`['your-app.onrender.com']`)
5. Create a **Background Worker** service pointing to the same repo:
   - Start Command: `celery -A gamekey_platform worker --loglevel=info`
   - Inject same environment variables (including `CELERY_BROKER_URL`).

---

4. Check for Understanding (10 min)

> ***Question**: "Why do the `web` and `worker` services use the same Docker image but different `command` values?"*
> ***Answer**: Both services run the same codebase and need the same environment variables, libraries, and settings. However, the `web` service runs the HTTP web server (`manage.py runserver` or Gunicorn) to handle API requests, while the `worker` service runs the Celery daemon (`celery worker`) to process background tasks. Using the same image simplifies building and keeps the dependencies identical.*

> ***Question**: "What's the problem with binding the Django dev server to `127.0.0.1` inside a Docker container?"*
> ***Answer**: `127.0.0.1` is the loopback interface, which is only accessible within the container itself. If the dev server is bound to `127.0.0.1`, the host machine and outer network cannot access the container's port. Binding to `0.0.0.0` tells the container to listen on all interfaces, allowing traffic from the host machine to reach the server.*

> ***Question**: "Why can't we use SQLite in production on Render?"*
> ***Answer**: SQLite stores data in a local file on the container's disk. Render containers have an ephemeral filesystem, meaning their disks are wiped and recreated on every deployment, restart, or scale event, resulting in complete database loss. Production environments require a persistent, remote database service (like PostgreSQL).*

> ***Question**: "What would happen if we forgot to add `python manage.py migrate` to the build command?"*
> ***Answer**: The application would start, but any database queries (such as listing games, user*

> *registration, or purchasing keys) would fail with database errors (e.g., `Relation does not exist` or `no such table`) because the database tables would not match the new code models.*

---

5. Final Integration Checklist (10 min)

Go through this final checklist with the class before wrapping up:

- [ ] **API Endpoints**: All endpoints return standard HTTP status codes (`200`, `201`, `400`, `401`, `403`, `404`).
- [ ] **Security**: Token Auth blocks unauthenticated writes and custom permissions protect publisher resources.
- [ ] **Async Engine**: The management command `check_expired_keys` successfully detects expired keys and dispatches background tasks.
- [ ] **Audit Trail**: Every webhook delivery attempt is logged in `WebhookDeliveryLog` showing response statuses and retries.
- [ ] **Tests**: The unit test suite passes successfully with code coverage ≥70%.
- [ ] **Containers**: Docker Compose spins up the web API, message queue, and Celery worker cleanly.

Project Structure (End of Day 10)

```
gamekey_platform/
├── gamekey_platform/
│   ├── __init__.py          # Loads Celery app
│   ├── celery.py
│   ├── settings.py
│   └── urls.py
├── games/
│   ├── management/
│   │   └── commands/
│   │       └── check_expired_keys.py
│   ├── migrations/
│   ├── tests/
│   │   └── test_webhook.py
│   ├── admin.py
│   ├── models.py            # Publisher, Game, GameKey, Order, OrderItem, WebhookDeliveryLog
│   ├── permissions.py
│   ├── serializers.py
│   ├── tasks.py             # Celery async webhook task
│   ├── viewsets.py
│   ├── views.py             # register, create_order
│   └── webhooks.py          # sync helper (Day 7 reference)
├── .env
├── .gitignore
├── docker-compose.yml
```

```
├── Dockerfile
├── manage.py
├── pytest.ini
└── README.md
```

API Reference — Sample Requests & Responses

All endpoints are prefixed with `/api/`. Authenticated endpoints require the header:

```
Authorization: Token <your-token>
```

POST `/api/register/`

Register a new user and receive an auth token.

Request

```
POST /api/register/
Content-Type: application/json

{
  "username": "dheeresh",
  "password": "securepass123"
}
```

Response `201 Created`

```
{
  "token": "9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a"
}
```

Response `400 Bad Request` — missing fields

```
{
  "error": "Username and password required."
}
```

GET `/api/publishers/`

List all publishers. Public endpoint (no auth needed).

Request

```
GET /api/publishers/
```

Response `200 OK`

```
[
  {
    "id": 1,
    "name": "Valve Corporation",
    "webhook_url": "https://valve.example.com/webhooks/gamekey",
    "user": 2
  },
  {
    "id": 2,
    "name": "CD Projekt Red",
    "webhook_url": "https://cdpr.example.com/hooks/expiry",
    "user": 3
  }
]
```

> `webhook_secret` *is always excluded from responses.*

POST `/api/publishers/`

Create a publisher profile. Requires auth.

Request

```
POST /api/publishers/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "webhook_secret": "mysecret_abc123",
  "user": 2
}
```

Response `201 Created`

```
{
  "id": 1,
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "user": 2
}
```

GET `/api/games/`

List all games. Public endpoint.

Request

```
GET /api/games/
```

Response `200 OK`

```
[
  {
    "id": 1,
    "title": "Half-Life 3",
    "publisher": 1,
    "price": "29.99"
  },
  {
    "id": 2,
    "title": "Cyberpunk 2077",
    "publisher": 2,
    "price": "49.99"
  }
]
```

POST `/api/games/`

Create a game. Requires auth.

Request

```
POST /api/games/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
```

```
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

Response `201 Created`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

GET `/api/games/{id}/`

Retrieve a single game.

Request

```
GET /api/games/1/
```

Response `200 OK`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

Response `404 Not Found`

```
{
  "detail": "No Game matches the given query."
}
```

PATCH `/api/games/{id}/`

Partially update a game. Requires auth + must be the publisher's owner.

Request

```
PATCH /api/games/1/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "price": "19.99"
}
```

Response `200 OK`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "19.99"
}
```

Response `403 Forbidden` — wrong owner

```
{
  "detail": "You do not have permission to perform this action."
}
```

POST `/api/orders/`

Buy a game key. Requires auth. Atomically assigns or generates a key with a 30-day expiry.

Request

```
POST /api/orders/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "game_id": 1
}
```

Response `201 Created`

```
{
  "order_id": 42,
  "game": "Half-Life 3",
```

```
  "key": "A3F9-B21C-4DE7-9901-CC84",
  "expires_at": "2026-07-16T10:30:00Z"
}
```

Response `400 Bad Request` — missing `game_id`

```
{
  "error": "game_id is required."
}
```

Response `404 Not Found` — game doesn't exist

```
{
  "error": "Game not found."
}
```

Response `401 Unauthorized` — no token provided

```
{
  "detail": "Authentication credentials were not provided."
}
```

Webhook Payload — `game_key.expired`

Sent asynchronously by Celery to the publisher's `webhook_url` when a key expires. Signed with HMAC-SHA256.

Headers

```
Content-Type: application/json
X-Signature: sha256=3c6e9b52a3c2ac3f7dca0c944b6d68b3f15ea2e3c0cf1b2e5f7a9d4c8e1f230b
```

Body

```
{
  "event": "game_key.expired",
  "game_key": "A3F9-B21C-4DE7-9901-CC84",
  "game_title": "Half-Life 3",
  "expired_at": "2026-07-16T10:30:00Z",
  "attempt": 0
}
```

> *`attempt` increments on each retry (max 3). Verify the signature on your end:*
>
> ```
> import hmac, hashlib, json
> expected = hmac.new(secret.encode(), json.dumps(payload, sort_keys=True).encode(), hashlib.sha
> assert hmac.compare_digest(f"sha256={expected}", request.headers["X-Signature"])
> ```

Webhook Retry Behaviour

Attempt	Delay before retry
0 → 1	60 seconds
1 → 2	120 seconds
2 → 3	240 seconds
3	Task marked failed, logged

All attempts (success or failure) are recorded in `WebhookDeliveryLog`.

Error Reference

Status	Meaning
`200 OK`	Successful read
`201 Created`	Resource created
`400 Bad Request`	Missing or invalid input
`401 Unauthorized`	No or invalid token
`403 Forbidden`	Authenticated but not the owner
`404 Not Found`	Resource does not exist

10 days · 1.5 hours/day · production-ready Django API with async webhooks